

THE PROGRAM AI ARCHITECTURE STANDARD

An open standard for running IT programs on AI — auditable, governed, and vendor-neutral

VERSION 1.1 DRAFT — REQUEST FOR COMMENTS

v1.0 published July 2026 · this draft is being revised in public from reviewer feedback · comment period open through October 1, 2026 · v1.1 final planned Q4 2026

Anthony Filipovich · Latitude Consulting

Released under Creative Commons Attribution 4.0 (CC BY 4.0) — share it, adapt it, implement it; keep the attribution.

Stop using AI randomly. Start thinking in systems.

Why this standard exists

Organizations are deploying AI agents faster than they are governing them. In the most recent industry surveys, 62 percent of organizations are experimenting with AI agents — yet in any given business function no more than 10 percent are scaling them, and only 39 percent report enterprise-level financial impact. Gartner expects over 40 percent of agentic AI projects to be cancelled within two years. Roughly four in five businesses are deploying or planning agents; fewer than half have any framework limiting what those agents may do unattended.

The failures are not model failures. They are architecture failures: numbers nobody can trace, agents nobody scoped, retrieval nobody governed, and autonomy nobody defined. Program management — the discipline that runs the largest, most expensive coordinated work in any enterprise — deserves better than improvised AI.

This document proposes a standard: ten verifiable conformance points and the reference architecture behind them, for running an IT program office on AI. It is vendor-neutral by construction — it names storage roles, open protocols, and controls, never products you must buy.

What this standard is not

It is not a tool recommendation, not a maturity model you can satisfy with a slide, and not a promise that AI runs your program. Judgment, accountability, political context, and final decisions remain human. The standard exists to make the AI layer underneath those humans auditable, governed, and worth trusting.

How to comment (this is a working document)

v1.0 was published deliberately as a **Request for Comments**, and this v1.1 draft is being revised in public as that feedback arrives. If you run programs, build enterprise AI, sit in security review, or audit any of the above — your objections are the value. Comment on the LinkedIn thread where you found this document, or open an issue in the public repository listed on the last page. Substantive contributions are credited by name in v1.1.

Most useful right now: Which conformance point would fail first in your organization — and why? What is missing? What would your CISO strike out?

Three terms used throughout

- **AI Pipeline** — a scheduled or event-triggered process running defined AI tasks (Extract, Classify, Score, Match, Generate) on defined data. Predictable, cheap, auditable.
- **AI Agent** — everything a pipeline does, plus two capabilities: **decide** (judgment against thresholds) and **act** (notify, route, escalate, write). Agents are where governance lives or dies.
- **The program office** — the function coordinating multiple projects toward one strategic outcome. Everything here applies at that cross-project level; single-project AI is a solved and lesser problem.

The Standard at a glance — the ten-point conformance test

A program office conforms to this standard when it can answer yes to all ten. Every no is a named gap with a section of this document that closes it. The test is designed to be self-assessed in one afternoon and audited in one day.

1	Every number an AI cites resolves to a SQL query in a system of record.
2	Every relationship claim resolves to a graph traversal — with validity intervals on the edges.
3	Every narrative claim carries provenance metadata a reader can click back to.
4	Every agent has a data contract: inputs, bundle, metadata, outputs, guardrails.
5	Every agent runs under its own least-privilege identity — no shared service accounts.
6	Every agent has a golden dataset and passed its evaluation gate before deployment.
7	Every agent action has a declared autonomy level, and promotions are eval-gated.
8	Agent telemetry is emitted in OpenTelemetry GenAI format into the observability platform you already run.
9	Data layers are exposed to agents via MCP; agent-to-agent coordination runs over A2A.
10	The controls above are mapped to ISO/IEC 42001 clauses or NIST AI RMF functions.

Points 1–3 govern what the AI is allowed to claim. Points 4–7 govern what an agent is allowed to do. Points 8–10 make the whole thing observable and defensible to the people who fund it. The rest of this document walks the architecture that produces the ten yeses.

Who owns this architecture

Three roles, and the distinction between them is what makes ownership real rather than ceremonial.

The Program Manager owns the architecture — accountable for what it produces, for its conformance to this standard, and for the decisions made on its outputs. Ownership here means what it means everywhere else in the role: PMs own budgets without being accountants and own delivery without writing code. Own ≠ build.

A build partner constructs it. The extraction pipelines, the context stack, and the data contracts (layers 2–3) are engineering work, done by engineers sitting close enough to the program to build against its reality rather than a generic spec. A PM “owning” an architecture with no build partner isn’t an owner — they’re a signature.

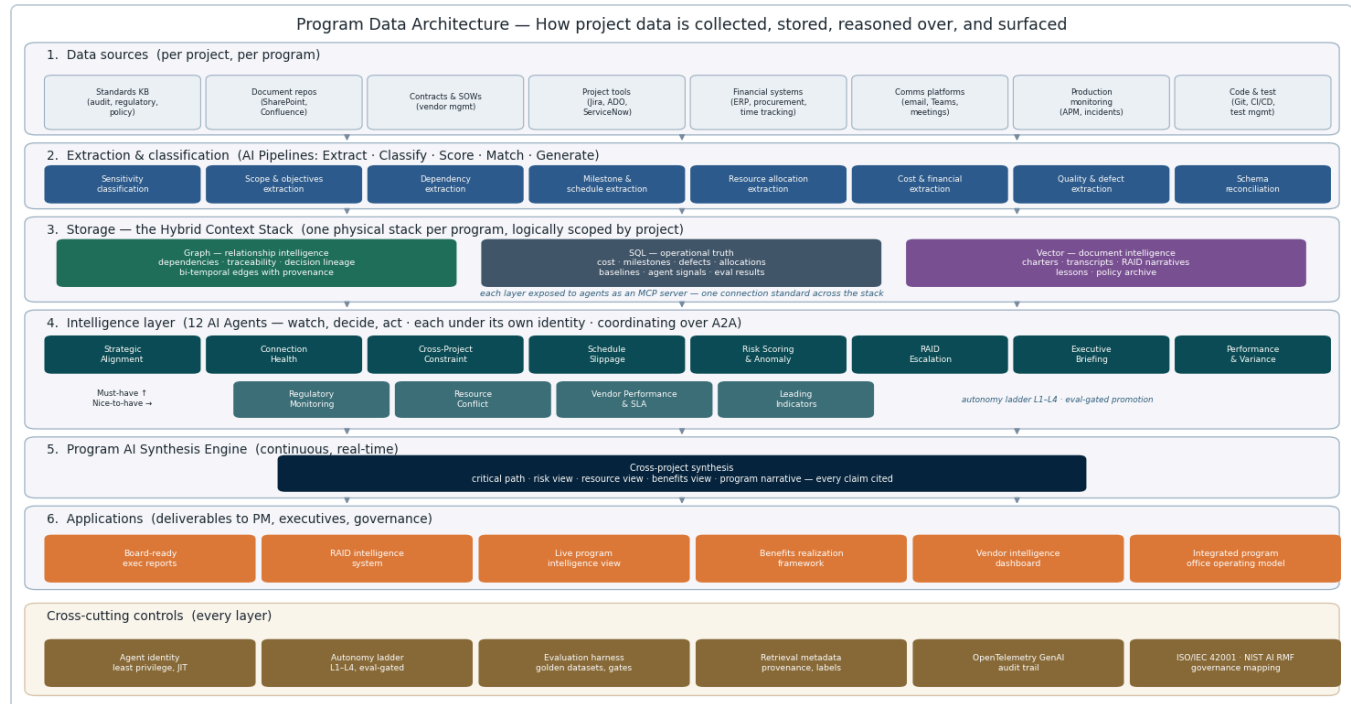
The PMO owns the standard, not the instances. Each program runs its own conforming stack, the way each program runs its own budget under one set of financial rules. Uniformity comes from conformance, not centralization.

And the guard that keeps ownership from decaying into rubber-stamping: **the PM verifies outputs, not pipes.** Every number resolves to a query in a system of record (conformance point 1); every claim carries provenance the reader can click back to (point 3). The PM never needs to read the code to check the number. If the PM can’t check it, the build isn’t done.

Added during the v1.0 comment period in response to reviewer feedback. Substantive contributors are acknowledged by name in v1.1; contributor credit is peer recognition of review contributions and does not constitute co-authorship. Feedback terms are on the last page and in CONTRIBUTING in the public repository.

The reference architecture

Six layers, each a distinct concern, plus the cross-cutting controls that apply at every layer. Data flows up; every claim that reaches an executive carries provenance back down.



Program Data Architecture — how project data is collected, stored, reasoned over, and surfaced

- **1 • Data sources.** Where program data already lives: project tools, documents, financials, communications, code and test systems, monitoring, contracts, standards.
- **2 • Extraction & classification.** AI pipelines that turn raw artifacts into structured, classified, sensitivity-labeled data. Five tasks: Extract, Classify, Score, Match, Generate.
- **3 • Storage — the Hybrid Context Stack.** Three logical roles: Graph for relationships, SQL for numbers and state, Vector for narrative. One physical stack per program, logically scoped per project.
- **4 • Intelligence.** Twelve bounded agents that watch, decide, and act — each with a contract, an identity, and an autonomy level.
- **5 • Synthesis.** The cross-project reasoning layer producing the program views: critical path, risk, resources, benefits, narrative — every claim cited.
- **6 • Applications.** What humans consume: executive reports, RAID intelligence, live program views, dashboards. Applications render intelligence; the stack owns it.

Principle 1 — The Hybrid Context Stack (conformance points 1–2)

Pure vector RAG — chunk the documents, embed them, retrieve by similarity — was the 2023 default and it fails program work three predictable ways. **Numbers hallucinate:** vector retrieval is fuzzy by design; ask it for the variance on a project and it returns “approximately \$2–3M” when the answer is \$2.34M — and executives discard AI the first time they catch a wrong number. **Relationships dissolve:** cross-project dependencies, scope-change cascades, and objective-to-deliverable traceability are graphs; embedding them approximates them. **State drifts:** “is this milestone approved?” is a deterministic question that deserves a deterministic answer, not a similarity score.

The standard therefore requires three storage roles, each doing what it is best at:

- **Graph — relationship intelligence.** Dependencies, traceability, decision lineage, stakeholder networks. Edges are bi-temporal: every edge carries a validity interval and a pointer to its source, and superseded edges are invalidated, never deleted — so the program can always answer “what did the dependency network look like when we approved this?”
- **SQL — operational truth.** Budgets, actuals, baselines, milestone dates, statuses, KPI values, agent signals, evaluation results. ACID, audit-ready, exact.
- **Vector — document intelligence.** Transcripts, charters, narratives, lessons learned, policy text. Semantic retrieval where semantics genuinely help.

These are **logical roles, not a vendor count**. Whether they run on three specialized databases or converge onto one or two platforms is a governance and operations decision; the standard fixes the responsibilities and the query discipline, never the products.

The seven rules

If it must be exact, use SQL. If it must be connected, use a graph. If it must be understood from text, use vector retrieval. If it must be explained with evidence plus relationships, combine them (GraphRAG). If it must connect, use MCP. If agents must coordinate, use A2A. If it must be done — an agent does it.

One more rule that prevents silent decay: **every entity declares one canonical tier** — its system of record. A risk is simultaneously a SQL row, a graph node, and a narrative; the other two are projections carrying provenance back to the canonical record, pipelines are the only writers of projections, and reconciliation checks run continuously. A cross-tier contradiction caught by an executive does the same damage as a hallucinated number.

Principle 2 — Provenance travels with everything (conformance point 3)

A retrieval without metadata is unauditible, and an AI recommendation that cannot be traced to authoritative, classified, current sources will be rejected by enterprise governance — rightly. Five metadata fields are non-negotiable on every fact, traversal, and chunk an AI touches:

- **Provenance.** Source system, document ID, ticket, or table-and-row. The reader must be able to click back to the original. No provenance, no defensibility.
- **Timestamp.** When this was generated or last modified. Agents reasoning on stale data produce confidently wrong answers; every contract declares how old its inputs may be.
- **Sensitivity label.** Public, Internal, Confidential, Restricted. Labels propagate: a summary built from Confidential inputs cannot flow to an Internal channel.
- **Confidence.** Similarity scores for vector results, validated-vs-inferred status for graph paths, data-quality status for SQL. Low confidence changes agent behavior — escalate, flag, or refuse.
- **Access policy.** Who may see this. The policy on an output is the most restrictive policy across its inputs.

Scope isolation is enforced, not assumed: row-level security in SQL, mandatory metadata filters on every vector query (cross-project semantic similarity is a leak channel), and node-level controls in the graph — the tier where access control is historically weakest and deserves the hardest look in security review.

Principle 3 — Open protocols, not proprietary glue (conformance point 9)

How agents connect to data, and to each other, stopped being a tool choice in 2025–2026 and became a protocol standard. This document specifies protocols precisely so the architecture stays portable across vendors:

- **MCP (Model Context Protocol)** connects agents to data and tools. Each stack layer is exposed as an MCP server — one standard interface for any agent from any vendor. MCP is natively supported across the major model providers, and a growing share of enterprise systems ship MCP endpoints out of the box, collapsing what used to be custom integration.
- **A2A (Agent2Agent)** connects agents to each other: capability cards, discovery, authentication, delegation. Governed by the Linux Foundation with 150+ supporting organizations. Whatever orchestration framework you run, the coordination rides an open protocol.

A standard made of storage roles, open protocols, and controls — with zero mandatory products — is a standard any company can adopt without procurement getting a vote first. That is deliberate.

Principle 4 — Agents are bounded and contracted (conformance point 4)

An agent is not a chatbot with ambitions. Every agent in the reference inventory is a five-line statement — **purpose, scope, stack reads, allowed actions, autonomy level** — and if any of the five is missing, the agent is not ready for production. Behind the five lines sits a data contract that specifies: the triggers that run it; the exact bundle it reads from each tier; the metadata that must travel with every retrieved item; the outputs it produces with reasoning chains attached; and the guardrails — what it must never do, what needs approval, what happens when its inputs are stale.

The reference inventory — twelve agents

Must-have (8): Strategic Alignment (watches objective-to-deliverable traceability), Connection Health (watches the system itself — integrations, drift, data quality), Cross-Project Constraint (dependency ripples), Schedule Slippage (slip risk and forecast dates), Risk Scoring & Anomaly (the math layer), RAID Escalation (the judgment layer), Executive Briefing (stakeholder-tailored narratives), Performance & Variance (baselines vs. actuals).

Nice-to-have (4): Regulatory Monitoring, Resource Conflict, Vendor Performance & SLA, Leading Indicators. Add these once the must-have layer produces reliable signals.

Two design rules that survive every review: agents that read but never write are pipelines misnamed as agents, and intelligence emerges from how narrow agents are wired together — never from making one agent omniscient. One monolithic reasoning call over everything destroys auditability, multiplies cost, and forfeits the per-domain specialization that makes outputs trustworthy.

Principle 5 — Every agent is an identity (conformance point 5)

Classic IAM knows two actors — humans and service accounts — and agents are neither. The standard requires every agent to run under its own first-class non-human identity: its own credentials, just-in-time scoped tokens matched to its data contract, lifecycle management, revocation at retirement. The escalation agent's token can read risk tables and write recommendations; it physically cannot touch the financial tables. This is what turns “bounded actions” from an aspiration into an enforced property — and it is how sensitivity labels actually propagate. Stepping stone for early adopters: one dedicated, scoped service account per agent. Never shared. Never borrowed from a human.

Principle 6 — The autonomy ladder (conformance point 7)

Autonomy is not binary, and treating it as binary is why most organizations deploying agents cannot answer the only question executives actually ask: *what can the AI do without asking?* Every agent action is assigned one of four levels, recorded in its contract:

L1 — Recommend	The agent drafts; a human executes. Day-one default for every agent.
L2 — Act with approval	The agent prepares the action in full; a human approves before it fires.
L3 — Act with notification	The agent executes; the accountable human is informed and can reverse.
L4 — Act autonomously	The agent executes within bounded scope; the audit trail is the oversight.

Promotion between levels is **earned, not configured**: an action moves up only after passing its evaluation gates over a declared period in service — and some actions are capped permanently (escalation to a steering committee stays at L2 no matter how reliable the agent). The ladder is one table. It replaces a hundred meetings.

Principle 7 — The evaluation harness (conformance point 6)

The evaluation harness is to agents what UAT is to software — the gate between built and deployed, and the discipline that separates surviving agentic projects from the 40 percent analysts expect to be cancelled. Four elements:

- **Golden dataset per agent.** Curated, versioned real inputs with known-correct outputs, in three layers: happy paths, edge cases, adversarial cases. Roughly a week of unglamorous work per agent — the cheapest insurance in this architecture.
- **Deployment gate.** Evals gate initial deployment, every model or prompt change, and every promotion up the autonomy ladder. A failing eval blocks the change.
- **Regression cadence.** Full re-runs at least weekly in production to catch drift, feeding the agent that watches the system itself.
- **LLM-as-judge for narratives.** Generated narratives are scored against a rubric by a second model, with human-reviewed samples keeping the judge calibrated.

Principle 8 — Observable, breakered, and governed (conformance points 8, 10)

Every agent action is logged with the agent identity, trigger conditions, and inputs — emitted in **OpenTelemetry GenAI** format, the vendor-neutral standard for tracing LLM calls, agent steps, tool calls, and token usage. The audit trail rides the observability platform the enterprise already runs; no new tooling category required.

Multi-agent chains get circuit breakers: a maximum chain depth before human review, per-item cooldowns so the same issue is not escalated hourly, and idempotent actions so a duplicate trigger cannot create duplicate tickets. One agent's misfire must never become an agent storm.

And the whole control set maps onto the two frameworks enterprise governance already speaks: **ISO/IEC 42001** (the certifiable AI management system standard) and the **NIST AI RMF** functions — Govern, Map, Measure, Manage. The contracts, ladder, harness, and audit trail are the operational evidence those frameworks ask for. Neither framework was designed for autonomous agents; the ladder and the harness are this standard's extension where the formal standards are still catching up.

Adopting the standard — Good, Better, Best

- **Good (months 1–3).** The SQL system of record you already have, plus lightweight vector search over your document repositories. One scoped service account per AI process. Start the golden datasets. Already eliminates the worst of manual status preparation.
- **Better (months 4–9).** Add the dependency graph (it can begin as relational tables), the first two or three contracted agents at L1–L2, MCP interfaces on the stack, and eval gates on everything that ships.
- **Best (months 12–24).** The full stack at both scopes, the twelve-agent inventory, per-agent identity, A2A coordination, OpenTelemetry GenAI end to end, and the ten-point test passing in an external audit.

Each level is a defensible stopping point that delivers value on its own. The ten-point test tells you exactly where you are at any moment — that is what makes it a standard rather than an aspiration.

What v1.1 needs from you

This standard is published as a Request for Comments because a standard nobody argued with is just an opinion. The comment period runs through **October 1, 2026**; v1.1 ships final in Q4 2026 with substantive contributors credited by name. The questions where disagreement is most valuable:

- Which conformance point would fail first in your organization — and is that a problem with your organization or with the point?
- Is ten the right number? What did we miss that your auditor, CISO, or steering committee would demand?
- Does the autonomy ladder need an L0 (AI may not touch this category at all)? Which actions belong there?
- For regulated industries: what would conformance evidence need to look like for your regulator?
- Graph access control is the weakest tier in practice. If you have solved node-level security at enterprise scale, the community needs your war story.

How to comment

Repository: github.com/Latitude-Consulting/program-ai-architecture-standard — open an Issue using the RFC Comment template; one issue per point of feedback. All change proposals go through Issues.

LinkedIn: comment on the post where you found this document — the thread is part of the record.

Direct: anthonyfilipovich@latitude.ai

Feedback terms — who owns what when you contribute — are on the next page.

Feedback license and ownership

Short version: your ideas make the standard better, your name goes on the standard, and the standard keeps a single copyright owner — which is what lets it be republished, licensed, and submitted to standards bodies without chasing signatures from every contributor.

By submitting feedback, comments, critiques, proposed language, or suggested edits to the Program AI Architecture Standard — via GitHub Issues, LinkedIn, direct message, email, or any other channel — you grant Anthony Filipovich / Latitude Consulting Inc. a perpetual, irrevocable, worldwide, royalty-free, non-exclusive license to incorporate, adapt, and publish those suggestions in current and future versions of the Standard and works derived from it (including under the Standard's CC BY 4.0 license).

Inclusion in the Contributors or acknowledgments list is **peer recognition of review contributions only** — it does not constitute co-authorship, co-ownership, or any transfer of copyright. The author remains the sole copyright owner of the work. If you do not agree to these terms, please do not submit feedback.

License and attribution

© 2026 Anthony Filipovich, Latitude Consulting Inc. Released under **Creative Commons Attribution 4.0 International (CC BY 4.0)**: you may share, adapt, and implement this standard, including commercially, provided you credit “The Program AI Architecture Standard, Anthony Filipovich / Latitude Consulting” and indicate changes. The author is the sole copyright owner; contributors are acknowledged for peer review. Full license: creativecommons.org/licenses/by/4.0

Selected sources

McKinsey, The State of AI (2025) and State of AI Trust in 2026 · Gartner agentic-AI forecasts (2025–2026) · Anthropic, Building Effective Agents (2024) · Linux Foundation, A2A Protocol (2025–2026) · Model Context Protocol, modelcontextprotocol.io · OpenTelemetry GenAI semantic conventions (2026) · Rasmussen et al., Zep: A Temporal Knowledge Graph Architecture for Agent Memory, [arXiv:2501.13956](https://arxiv.org/abs/2501.13956) · ISO/IEC 42001 · NIST AI Risk Management Framework · EU AI Act implementation timeline, artificialintelligenceact.eu. Full citations appear in the companion Course Reference.

Stop using AI randomly. Start thinking in systems.